

Trabajo Final Integrador - Desarrollo de Aplicaciones Web

Fase 4

Infraestructura, DevOps y Ambientes

Sistema de gestion de canchas y matchmaking deportivo

Version de trabajo basada en las Fases 1, 2 y 3 del proyecto

Resumen ejecutivo

La Fase 4 define como desplegar en la nube la arquitectura evolucionada en Fase 3. El objetivo es transformar el diseño de microservicios en una plataforma operable, con ambientes separados, CI/CD, contenedores, Kubernetes, Terraform, observabilidad y una estimación de costos.

La propuesta utiliza AWS como nube de referencia y mantiene coherencia con el sistema ya trabajado: API Gateway, servicios por dominio, RabbitMQ, Saga Orchestrator, Query Service, Redis y PostgreSQL con replicas de lectura.

Arquitectura de infraestructura propuesta

La arquitectura de infraestructura ubica el frontend React en S3 y CloudFront, publica la entrada HTTP mediante un Application Load Balancer y ejecuta los microservicios dentro de EKS. Los componentes administrados reducen carga operativa y permiten escalar por servicio.

Fase 4 - Infraestructura AWS propuesta

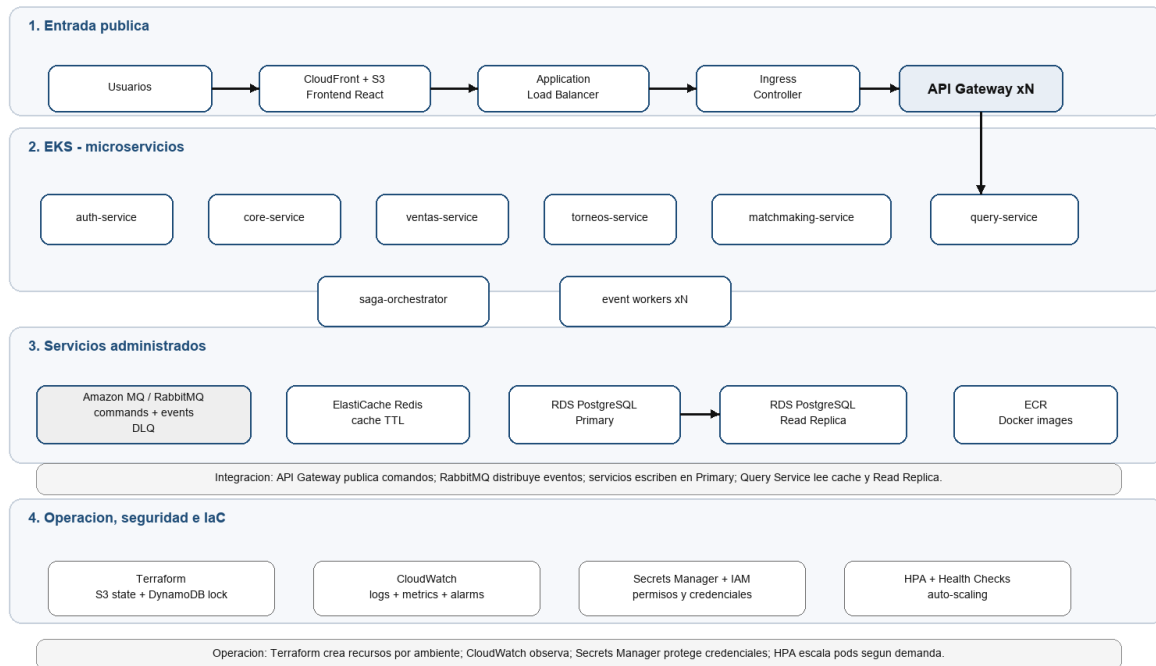


Figura 1. Infraestructura AWS propuesta para desplegar la arquitectura de Fase 3.

Separación de ambientes

Ambiente	Propósito	Rama/Fuente	Validaciones
Desarrollo	Trabajo local y sandbox por feature	feature/*	Lint, unit tests, build Docker local
Testing/QA	Pruebas funcionales automatizadas	develop	Unit tests, integration tests, Postman/Newman
UAT	Smoke tests y validación de stakeholders	release/*	Smoke tests, reservas, pagos y matchmaking
Producción	Usuarios reales	main + tag	Deploy sin downtime, rollback y monitoreo

Branching y flujo de promoción

Se propone GitFlow porque el enunciado requiere ambientes progresivos y UAT. El flujo feature -> develop -> release -> main permite separar trabajo en curso, validación funcional, versión candidata y producción.

- feature/* se usa para desarrollo local y cambios aislados.
- develop despliega a Testing/QA.
- release/* despliega a UAT con aprobación manual.
- main representa producción y se asocia a tags versionados.

Pipeline CI/CD

GitHub Actions automatiza la integracion y el despliegue. El pipeline funciona como filtro de calidad antes de promover una version entre ambientes.

Fase 4 - Pipeline CI/CD y ambientes

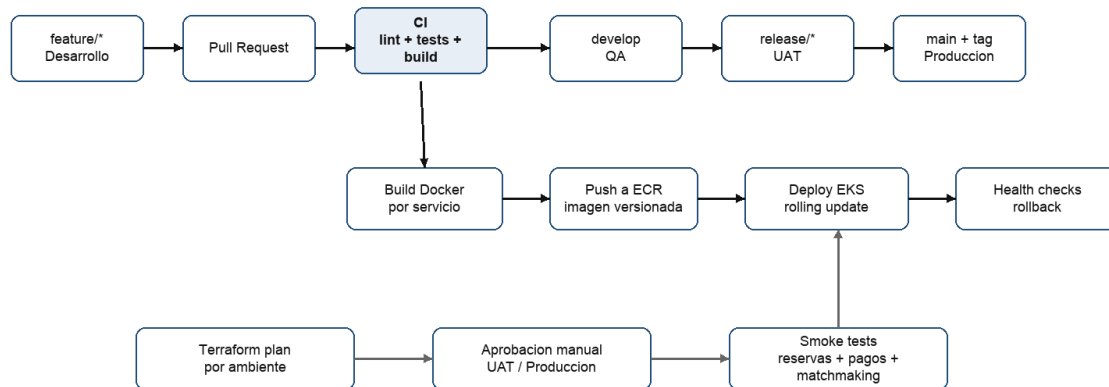


Figura 2. Pipeline CI/CD propuesto y relacion con ambientes.

- CI: checkout, dependencias, lint, unit tests, integration tests, build Docker y escaneo basico.
- CD QA: despliegue automatico desde develop.
- CD UAT: despliegue desde release/* con aprobacion manual.
- CD Produccion: despliegue desde main/tag con rolling update y rollback.

Docker, Kubernetes y despliegue

Cada microservicio se empaqueta como imagen Docker independiente y se publica en ECR. En Kubernetes cada servicio se modela con Deployment, Service, ConfigMap, Secret y HPA cuando corresponde.

Recurso Kubernetes	Uso en el proyecto
Deployment	Define replicas, imagen y estrategia de rollout por microservicio.
Service	Permite comunicacion interna estable entre servicios.
Ingress	Expone el API Gateway hacia el ALB.
ConfigMap	Guarda configuracion no sensible por ambiente.
Secret	Guarda credenciales, JWT secrets y URLs privadas.
HorizontalPodAutoscaler	Escala pods segun CPU, memoria o metricas de carga.

Infraestructura como Codigo

Terraform define la infraestructura de manera reproducible. Se proponen modulos reutilizables y carpetas por ambiente para cambiar escala, politicas y costos sin duplicar diseno.

Estructura propuesta:

```
infra/
modules/
network/
eks/
rds-postgres/
elasticsearch-redis/
rabbitmq/
observability/
frontend-static/
envs/
dev/
qa/
uat/
prod/
```

El estado remoto se guarda en S3 y se bloquea con DynamoDB para evitar modificaciones simultaneas.

Observabilidad y operacion

- CloudWatch centraliza logs, metricas y alarmas.
- Se monitorea latencia del API Gateway, errores 4xx/5xx, CPU/memoria por pod y reinicios.
- RabbitMQ se controla por mensajes pendientes, errores y DLQ.
- RDS se controla por CPU, conexiones, almacenamiento y replica lag.
- Redis se controla por memoria, expiraciones y hit rate.
- Las sagas fallidas generan alertas para revisar compensaciones.

Estimacion de costos AWS

La siguiente estimacion corresponde a un ambiente productivo inicial. QA y UAT pueden reducir costos con tamanos menores, namespaces compartidos o apagado programado.

Componente	Servicio AWS	Cantidad	Rol	Costo mensual aprox.
Load Balancer	ALB	1	Entrada HTTP/HTTPS	USD 25
Kubernetes control plane	EKS	1 cluster	Orquestacion	USD 73
Nodos de aplicacion	EC2 t3.medium	2 nodos	Pods de microservicios	USD 65
Container registry	ECR	1 repo por servicio	Imagenes Docker	USD 5
Base primaria	RDS PostgreSQL db.t3.medium	1	Escrituras	USD 70
Read replica	RDS PostgreSQL db.t3.medium	1	Lecturas	USD 70
Cache	ElastiCache Redis t4g.small	1	Cache y datos temporales	USD 30
Broker	Amazon MQ RabbitMQ	1	Comandos, eventos y DLQ	USD 35
Frontend	S3 + CloudFront	1	React build y CDN	USD 10
Observabilidad	CloudWatch	1	Logs, metricas y alarmas	USD 25

Componente	Servicio AWS	Cantidad	Rol	Costo mensual aprox.
Backups/state	S3 + DynamoDB	1	Backups y Terraform state	USD 20
Total estimado	-	-	-	USD 428

Justificacion tecnica y de negocio

Desde lo tecnico, EKS permite operar microservicios con replicas, health checks y escalado; Terraform evita configuracion manual irrepetible; GitHub Actions automatiza validaciones y despliegues; RDS, Redis y RabbitMQ administrados reducen carga operativa.

Desde el negocio, la propuesta protege operaciones centrales como reservas, pagos y matchmaking. La separacion de ambientes reduce riesgo, los despliegues tienen rollback y el costo inicial se mantiene razonable para una escala temprana.

Resultado esperado

- Ambientes separados y trazables.
- Pipeline CI/CD con gates de calidad.
- Imagenes Docker versionadas por servicio.
- Kubernetes como plataforma de despliegue.
- Terraform para infraestructura reproducible.
- Observabilidad y alertas operativas.
- Costos cloud estimados y defendibles.

Fuentes revisadas

- Enunciado del Trabajo Final Integrador de Desarrollo Web.
- Documento base recibido con version previa de Fase 4.
- Documentacion local de Fase 3 sobre robustez y escalabilidad.
- Estructura del repositorio con microservicios y dependencias de mensajeria, cache y circuit breaker.